Middleware Architectures 2

Lecture 7: Kubernetes Storage

doc. Ing. Tomáš Vitvar, Ph.D.

tomas@vitvar.com • @TomasVitvar • https://vitvar.com

Czech Technical University in Prague

Faculty of Information Technologies • Software and Web Engineering • https://vitvar.com/lectures

Modified: Sun Apr 26 2026, 12:29:29

Humla v1.0

Overview

- Storage Fundamentals
  - *Introduction to Storage in Kubernetes*
  - *Core Storage Abstractions*
- Storage Provisioning
- Storage Backends

Why Persistent Storage Matters

- Containers are **ephemeral** by design
  - *When a container stops or restarts, its filesystem is discarded*
  - *Data written inside a container is lost on Pod deletion or reschedule*
- Many real-world applications require persistent data
  - *Databases (MySQL, PostgreSQL, MongoDB)*
  - *Message queues (Kafka, RabbitMQ)*
  - *File uploads, caches, configuration stores*
- Kubernetes must provide a way to decouple data lifecycle from Pod lifecycle
  - *Data must outlive the Pod that created it*
  - *Storage must be accessible when a Pod is rescheduled to another node*
  - *Multiple Pods may need to share the same data*

Storage Challenges

- Data persistence
  - *Default container storage is tied to the container runtime layer (OverlayFS)*
  - *All writes are lost when a container is removed*
- Portability across nodes
  - *Pods can be rescheduled to any node in the cluster*
  - *Node-local storage is not available after rescheduling*
- Multi-tenancy and isolation
  - *Different applications must not access each other's data*
  - *Storage should be namespaced and access-controlled*
- Diverse storage backends
  - *Teams use different storage solutions: NFS, iSCSI, cloud block/file storage*
  - *Kubernetes must abstract over all of them through a common API*
- Shared access
  - *Some workloads need multiple Pods to read/write the same volume simultaneously*
  - *Access semantics differ across storage backends (block vs. file vs. object)*

Kubernetes Storage Model Overview

- Kubernetes provides a layered abstraction for storage
  - *Applications request storage without knowing the underlying backend*
  - *Administrators configure and expose storage resources*
- Key objects in the storage model
  - ***Volume** – ephemeral or persistent storage attached to a Pod*
  - ***PersistentVolume (PV)** – a storage resource in the cluster*
  - ***PersistentVolumeClaim (PVC)** – a request for storage*
  - ***StorageClass** – a template for dynamically provisioning PVs*
  - ***CSI Driver** – plugin interface to external storage systems*
- Two provisioning models
  - ***Static** – administrator creates PVs manually in advance*
  - ***Dynamic** – Kubernetes creates PVs automatically on demand*

Overview

- Storage Fundamentals
  - *Introduction to Storage in Kubernetes*
  - *Core Storage Abstractions*
- Storage Provisioning
- Storage Backends

Volumes

- A **Volume** is a directory accessible to containers in a Pod
  – *Declared in the Pod spec; mounted into one or more containers*
  – *Lifecycle is tied to the Pod (not the individual container)*
  – *Data in a volume survives container restarts within the same Pod*
- Common ephemeral volume types
  – `emptyDir` – *scratch space created when Pod starts; deleted with Pod*
  – `configMap` / `secret` – *inject configuration or credentials as files*
  – `downwardAPI` – *expose Pod metadata to containers as files*
- Usage example

```
spec:
  volumes:
  - name: cache-vol
    emptyDir: {}
  containers:
  - name: app
    image: myapp:1.0
    volumeMounts:
    - mountPath: /cache
      name: cache-vol
```


PersistentVolume (PV)

- A **PersistentVolume** is a cluster-level storage resource
  – *Provisioned by an administrator or dynamically by a StorageClass*
  – *Independent lifecycle from any individual Pod*
  – *Backed by a real storage system (NFS, cloud disk, local path, etc.)*
- Key PV properties
  – ***Capacity*** – *storage size (e.g.* `10Gi`*)*
  – ***Access Modes*** – *how the volume can be mounted (see next slide)*
  – ***Reclaim Policy*** – *what happens when the PV is released*
  – ***Volume Mode*** – `Filesystem` *(default) or* `Block`
  – ***StorageClass*** – *class used for binding and provisioning*
- PV phases
  – `Available` ⟶ `Bound` ⟶ `Released` ⟶ `Failed`

PV Access Modes and Reclaim Policies

- Access Modes
  - `ReadWriteOnce (RWO)` – *mounted read-write by a single node*
  - `ReadOnlyMany (ROX)` – *mounted read-only by nodes*
  - `ReadWriteMany (RWX)` – *mounted read-write by nodes simultaneously*
  - `ReadWriteOncePod (RWOP)` – *read-write by a single Pod*
- Not all backends support all access modes
  - *Block storage (AWS EBS, GCE PD) typically supports only* `RWO`
  - *Shared filesystems (NFS, Azure Files) support* `RWX`
- Reclaim Policies
  - `Retain` – *PV is kept after PVC deletion; manual cleanup required*
  - `Delete` – *PV and underlying storage are deleted with the PVC*

PersistentVolumeClaim (PVC)

- A **PersistentVolumeClaim** is a request for storage by a user
  - *Declares required capacity, access mode, and a StorageClass*
  - *Kubernetes binds the PVC to a matching PV*
  - *The bound PV is then mountable in a Pod*
- PVC is namespace-scoped; PV is cluster-scoped
- Example PVC and Pod using it

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: standard
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: my-pvc
  containers:
```

StorageClass

- A **StorageClass** defines a template for dynamic PV provisioning
  - *Specifies the provisioner (CSI driver or in-tree plugin)*
  - *Carries parameters specific to the storage backend*
  - *Defines the default reclaim policy for dynamically created PVs*
- Benefits
  - *Different classes can represent different tiers: fast SSD vs. slow HDD*
  - *Developers request storage by class name without knowing the backend*
  - *One cluster can offer multiple classes for different use cases*
- Example

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```


Overview

- Storage Fundamentals
- Storage Provisioning
  - *Static Provisioning*
  - *Dynamic Provisioning*
- Storage Backends

What is Static Provisioning?

- Administrator creates PersistentVolumes **manually** in advance
  - *The underlying storage resource must already exist*
  - *PV spec describes how to connect to the resource (NFS share, disk path, etc.)*
- Workflow
  1. *Admin provisions storage resource (e.g. creates NFS export)*
  2. *Admin creates a PV object pointing to that resource*
  3. *Developer creates a PVC requesting matching properties*
  4. *Kubernetes binds the PVC to the existing PV*
  5. *Pod mounts the volume via the PVC*
- Good for
  - *Small clusters or on-premise setups with fixed storage infrastructure*
  - *Workloads with specific, pre-configured storage requirements*
  - *Environments where an administrator controls every storage resource*

Static Provisioning Example

- Admin creates a PV backed by an NFS share

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 20Gi
  accessModes:
  - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    server: 192.168.1.100
    path: /exports/data
```

- Developer creates a matching PVC

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
  - ReadWriteMany
  resources:
    requests:
      storage: 20Gi
```

- Kubernetes finds the PV that satisfies the PVC criteria and binds them

Static Provisioning – Pros and Cons

- Pros
  - *Full administrator control over every storage resource*
  - *Works with any storage backend regardless of CSI support*
  - *Predictable: storage is prepared before workloads request it*
  - *Regulated environments requiring manual approval workflows*
- Cons
  - *Requires manual effort for every new storage resource*
  - *Does not scale well in large or dynamic clusters*
  - *Risk of PV/PVC mismatch causing pending claims*
  - *Capacity may be under-utilized if not carefully matched*
  - *No automation – developers must wait for admin action*

Overview

- Storage Fundamentals
- Storage Provisioning
  - *Static Provisioning*
  - *Dynamic Provisioning*
- Storage Backends

What is Dynamic Provisioning?

- Kubernetes automatically creates a PV when a PVC is submitted
  - *PVC references a **StorageClass** instead of a specific PV*
  - *The StorageClass provisioner calls the storage backend API*
    → The volume is allocated
  - *The PV is created, bound to the PVC, and ready for use*
- Workflow
  1. *Administrator deploys a StorageClass*
     - *Defines the provisioner (e.g. CSI driver), parameters for the backend*
  2. *Developer creates a PVC referencing the StorageClass*
  3. *Kubernetes provisioner creates a new PV on the backend*
  4. *PVC is bound to the new PV; Pod can mount it immediately*
- Supported by all major cloud providers and on-premise solutions
  - *AWS EBS, GCE PD, Azure Disk, OpenEBS, etc.*

Dynamic Provisioning Example

- StorageClass for AWS EBS using the CSI driver

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: aws-ebs-sc
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

- Developer creates a PVC (PV does not exist)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: aws-ebs-sc
  resources:
    requests:
      storage: 10Gi
```

- Kubernetes calls the EBS CSI driver
  - *new EBS volume is created and bound*

Dynamic Provisioning – Pros and Cons

- Pros
  - *Self-service for developers – no admin intervention required*
  - *Scales automatically with cluster demand*
  - *Eliminates idle pre-provisioned storage*
  - *Integrates with cloud cost management (pay-per-use)*
  - *Supports version control of storage configuration via StorageClass*
- Cons
  - *Requires a working CSI driver or in-tree provisioner*
  - *Less control over individual storage resources*
  - *Uncontrolled PVC creation can lead to unexpected cloud costs*
  - *Debugging provisioning failures can be complex (CSI logs, events)*
  - *Not always available in air-gapped or strictly controlled environments*

Overview

- Storage Fundamentals
- Storage Provisioning
- Storage Backends
  - *Local Node Storage*
  - *External Storage*

hostPath Volumes

- Mounts a file/directory from the **node's filesystem** into the Pod
  - *Directly exposes a node path such as `/var/log` or `/data`*
  - *The container reads and writes directly to the node's disk*
- Common use cases
  - *Log collection agents that need access to `/var/log`*
  - *Monitoring tools accessing `/proc` or `/sys`*
  - *Development and testing on single-node clusters*
- Risks and limitations
  - *Tightly couples the Pod to a specific node*
  - *If the Pod is rescheduled to another node, the data is not available*
  - ***Security risk**: container can read/modify sensitive host files*
  - *Not suitable for multi-node production workloads*
- This should generally be avoided for persistent application data in production

Local Persistent Volumes

- A **local** PersistentVolume uses a dedicated disk or partition on a node
  - *Defined with `local` volume type in the PV spec*
  - *Unlike `hostPath`, local PVs support standard PV/PVC lifecycle*
  - *Kubernetes topology-aware scheduling ensures Pods land on the correct node*
- Example

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv
spec:
  capacity:
    storage: 100Gi
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-storage
  local:
    path: /mnt/disks/ssd1
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
          values:
          - worker-node-1
```

Local Node Storage – Pros and Cons

- Pros
  - *Very high I/O performance – direct access to local NVMe or SSD disks*
  - *Low latency; no network overhead*
  - *Simple setup; no external storage system required*
  - *Good for read-heavy workloads, caches, and stateful databases*
- Cons
  - *Data is bound to a specific node – no built-in replication*
  - *Node failure means data loss unless application handles replication itself*
  - *Pod scheduling must respect node affinity – reduces scheduling flexibility*
  - *Scaling out requires adding disks to specific nodes manually*
  - `hostPath` *is insecure and unsuitable for multi-tenant clusters*
- Recommended pattern: use local volumes only with applications that manage their own data replication (e.g. distributed databases)

Overview

- Storage Fundamentals
- Storage Provisioning
- Storage Backends
  - *Local Node Storage*
  - *External Storage*

Network File System (NFS)

- NFS is a widely used network-attached filesystem
  - *A central NFS server exports directories shared over the network*
  - *Multiple Pods can mount the same NFS share simultaneously (*`ReadWriteMany`*)*
  - *Supported natively in Kubernetes via the* `nfs` *volume type*
- Typical setup
  - *NFS server runs on a dedicated machine or network appliance*
  - *Kubernetes nodes mount NFS exports; data is network-accessible from all nodes*
  - *Dynamic provisioning available via the* `nfs-subdir-external-provisioner`
- Example PV using NFS

```
spec:
  capacity:
    storage: 50Gi
  accessModes:
  - ReadWriteMany
  nfs:
    server: nfs.example.com
    path: /exports/shared
```

- Common for shared content, logs aggregation, and legacy workloads

Cloud Block Storage and CSI

- Cloud providers offer managed block storage backends
  - ***AWS**: Elastic Block Store (EBS) – provisioned via* `ebs.csi.aws.com`
  - ***GCP**: Persistent Disk (PD) – provisioned via* `pd.csi.storage.gke.io`
  - ***Azure**: Managed Disk / Azure Files – via* `disk.csi.azure.com`
- **Container Storage Interface (CSI)**
  - *Standard API between Kubernetes and storage plugins*
  - *Decouples storage drivers from the Kubernetes core*
  - *CSI driver is deployed as a DaemonSet/Deployment in the cluster*
  - *Exposes capabilities: provision, attach, mount, snapshot, resize, clone*
- CSI driver operations
  - `CreateVolume` / `DeleteVolume` *– lifecycle management*
  - `NodePublishVolume` / `NodeUnpublishVolume` *– mount/unmount on node*
  - `CreateSnapshot` *– point-in-time backup*
- On-premise options: Ceph, iSCSI, OpenEBS

Overview

- Storage Fundamentals
- Storage Provisioning
- Storage Backends
  - *Advanced Topics and Best Practices*

StatefulSet with Persistent Storage

- **StatefulSet** is the workload resource designed for stateful applications
  - *Each Pod gets a stable identity (name, DNS) and its own PVC*
  - *PVCs are created from a* `volumeClaimTemplates` *section*
  - *PVCs survive Pod deletion and are reattached when the Pod is recreated*
- Example

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  template:
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: 20Gi
```

- Each Pod (e.g. `mysql-0`, `mysql-1`) gets its own `data-mysql-N` PVC

Volume Snapshots

- Kubernetes supports point-in-time snapshots of PersistentVolumes
  – *Requires a CSI driver that implements snapshot capabilities*
  – *Snapshots are managed via* `VolumeSnapshot`, `VolumeSnapshotContent`, *and* `VolumeSnapshotClass` *resources*
- Use cases
  – *Backup and disaster recovery*
  – *Creating test environments from production data*
  – *Pre-upgrade checkpoints*
- Creating a snapshot

```yaml
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: db-snapshot
spec:
  volumeSnapshotClassName: csi-aws-vsc
  source:
    persistentVolumeClaimName: db-pvc
```


Storage Best Practices

- Choose the right storage type for the workload
  – *Use local volumes only for applications that replicate data themselves*
  – *Use shared (RWX) storage only when multiple Pods need concurrent access*
  – *Default to cloud block storage (CSI, RWO) for most stateful workloads in cloud*
- Use dynamic provisioning wherever possible
  – *Reduces operational overhead and scales automatically*
  – *Define StorageClasses for different performance tiers*
- Set appropriate reclaim policies
  – *Use* `Retain` *for production data to prevent accidental deletion*
  – *Use* `Delete` *for ephemeral or dev/test environments*
- Enable volume snapshots for critical workloads
- Apply resource quotas on PVC requests per namespace to control costs
- Monitor storage utilization; avoid PV over-provisioning